COMPUTER NETWORKS LABORATORY PROJECT

LAN-Based Multi-User Collaborative Whiteboard

A Resilient Full-Stack Vector Canvas & WebSocket Synchronization System for Local Subnets and Air-Gapped Networks

Roll Number	Student Name	Assigned Engineering Specialization
CS24B1025	Anirudh	Network Protocol Architecture, Node.js WebSocket Server & Berkeley Socket Framing
CS24B1097	Krishna Chaitanya	React 18 Vector Canvas Engine, 2D Affine Transforms & Synchronized Geometry UI
CS24B1108	Harish	Integrated Real-Time Group Chat, SQLite WAL Persistence & Session Code Management
CS24B1106	Manas Vivek	Concurrency Control, 4-Dimensional RBAC Matrix, Milestone Sandboxing & LAN Testing

Executive Summary & Abstract

Modern collaborative digital workspaces (such as Miro, Figma FigJam, and Google Jamboard) rely heavily on remote cloud infrastructures. While feature-rich, their mandatory reliance on wide-area network (WAN) connectivity introduces severe drawbacks for academic research, industrial prototyping, and defense laboratories: high round-trip latency, dependency on public cloud availability, substantial WAN bandwidth consumption, and security vulnerabilities associated with uploading technical designs to third-party infrastructure. This project presents the design, network protocol engineering, and empirical realization of a production-grade **LAN-Based Multi-User Collaborative Whiteboard**. Engineered with a Node.js full-duplex WebSocket server, an embedded SQLite database engine (WAL mode), and a high-performance React 18 Canvas 2D vector renderer, the application operates seamlessly over air-gapped local area networks (LANs), Wi-Fi routers, and mobile hotspots with **zero external Internet dependency**, while seamlessly supporting Internet-wide collaboration when cloud tunneling is desired.

Key capabilities include deterministic sub-5 millisecond drawing synchronization, an optimistic local rendering pipeline with server-arbitrated monotonic sequencing (seq), complete vector state representation, conflict-free user-scoped undo/redo stacks, dynamic 4-dimensional Role-Based Access Control (RBAC), isolated instance milestone sandboxing, dynamic interactive tables, a geometric hit-testing vector element eraser, in-app group chat, and an automated UDP subnet discovery engine. Critically, end users operate the software through a true **Single-Click Executable (`Collaborative_Whiteboard.exe`)** that launches the full working whiteboard directly with zero terminal usage.

Parameter	Specification / Engineering Implementation Details
Transport Protocol	Full-Duplex WebSockets over TCP/IP (RFC 6455) with TCP_NODELAY enabled
Subnet Peer Discovery	UDP Broadcast datagrams on Port 45455 / 41234 for zero-configuration LAN host detection
Default Network Ports	TCP Port 5000 (HTTP REST & WebSocket Gateway); UDP Port 45455 (Subnet Discovery)
Backend Architecture	Node.js Asynchronous Event Loop + Express HTTP API + SQLite Embedded DB (WAL mode)
Frontend Canvas Engine	React 18 + HTML5 Canvas 2D with 2D Affine Transformation Matrix (Zoom 15%–400%, Pan)
State Representation	Mathematical Vector Objects (strokes, polygons, arrows, tables, text) with tombstone consistency
Security & Governance	4-Letter Alphanumeric Session Codes (e.g. SPZG) + Optional Room PIN + 4-Dimensional RBAC Matrix
Execution Model	Zero-Terminal 1-Click Startup: Collaborative_Whiteboard.exe directly opens full application

Documentation Structure Overview

This formal technical documentation report provides an exhaustive reference complementary to the final presentation: Section 1 details problem statement and scope; Section 2 analyzes system architecture and network design; Section 3 formalizes socket communications and wire message schemas; Section 4 details real-time synchronization and the vector eraser engine; Section 5 addresses concurrency control; Section 6 analyzes state recovery and persistence; Section 7 covers Role-Based Access Control; Section 8 provides an exhaustive **User Guide** detailing the 1-Click executable workflow and all canvas tools; and Section 9 provides empirical laboratory benchmarks and individual team member contributions.

1. Problem Statement, Objectives, and System Scope

1.1 Background and Educational Context

In modern computer science education and engineering laboratory environments, collaborative diagramming, protocol tracing, and system architecture sketches are essential communication workflows. Traditional instructional physical whiteboards severely restrict participation to a single presenter standing at the front, precluding simultaneous multi-student contributions. While commercial cloud whiteboards offer multi-user drawing, their mandatory Internet requirement renders them unusable in offline laboratory exams, air-gapped security enclaves, or network environments facing institutional firewall restrictions.

1.2 Critical Limitations of Commercial Cloud Solutions

• **Mandatory WAN Dependency:** Cloud tools require uninterrupted WAN internet connectivity. Any gateway hiccup or DNS timeout aborts active collaboration.
• **High Round-Trip Latency:** Packets routed through distant third-party cloud data centers incur 80–150ms round-trip latency, causing noticeable cursor lag.
• **Data Sovereignty Risks:** Proprietary code diagrams and examination solutions are transmitted to external servers, violating academic integrity and data sovereignty policies.
• **WAN Bandwidth Bottlenecks:** Streaming continuous canvas updates across dozens of laboratory nodes congests campus upstream bandwidth.

1.3 Project Motivation: Autonomous Local Area Network Collaboration

The motivation of this project is to develop a self-contained, low-latency, resilient collaborative whiteboard system operating natively over the Local Area Network (LAN). By utilizing direct WebSocket connections over TCP/IP within the local subnet (e.g. 192.168.1.0/24 or 10.x.x.x), the system achieves sub-5ms event dispatching, zero campus bandwidth usage, complete data confidentiality, and 100% functionality without internet access.

1.4 Primary Engineering Objectives

1. **Zero-Internet Autonomous Operation:** Execute full room hosting, client discovery, vector drawing, and chat over physical LAN switches without internet.
2. **High-Fidelity Real-Time Synchronization:** Deliver vector stroke broadcasts with deterministic sub-10ms latency across concurrent participants.
3. **Full Vector Toolkit:** Support freehand ink, geometric shapes, dynamic tables, text annotations, multi-select marquee drag, snap screenshot crop, and true vector erasing.
4. **State Persistence & Late-Joiner Hydration:** Guarantee that any client joining mid-session immediately receives the entire historical canvas state.
5. **Deterministic Governance & Sandboxing:** Provide a 4-dimensional RBAC matrix and isolated milestone sandboxes for private prototyping.

1.5 Operational Scope & Boundary Matrix

In-Scope Functional Capabilities	Out-of-Scope / Non-Goals
• Full-duplex WebSocket communication over LAN TCP/IP • Subnet peer discovery via UDP broadcast (Port 45455) • Discrete vector object modeling & point-to-segment eraser • Persistent SQLite database storage with WAL journal mode • 4-Dimensional RBAC policy enforcement (Draw, Chat, Instances, Public) • Milestone snapshot isolation & live sandbox broadcast • Standalone zero-terminal executables for Server and Client	• Proprietary cloud OAuth authentication gates • Lossy UDP-based stroke transmission (TCP required for canvas fidelity) • Heavy video/audio conferencing streaming (VoIP) • Raster-only bitmap canvas storage (vector format enforced)

2. Detailed System Architecture and Network Design

2.1 Centralized Star-Topology Architectural Model

The collaborative whiteboard is architected upon a centralized star topology. A central host runs the backend server process, coordinating all active rooms and acting as the authoritative arbitrator of canvas actions, sequence numbers, and chat events. Clients establish direct full-duplex TCP WebSocket connections to the central server. Peer-to-peer (P2P) mesh topologies were deliberately avoided due to quadratic network overhead O(N^2) socket connections and complex distributed consensus synchronization failures across unmanaged local subnets.

2.2 Multi-Tier System Architecture

The system comprises three cleanly decoupled architectural tiers:
1. Presentation Layer (React 18 + HTML5 Canvas): Handles user interaction, mouse/touch event sampling, 2D affine coordinate transformations (world-to-screen and screen-to-world), optimistic local drawing, zoom (15%–400%), smooth grid rendering, and client-side bounding box collision detection.
2. Network & Arbitration Layer (Node.js WebSocket Server): Manages connection lifecycle, room isolation maps (roomSockets), monotonic sequence stamping (seq), RBAC permission filtering, and high-frequency JSON message fan-out.
3. Persistence Layer (SQLite Database): Uses SQLite with Write-Ahead Logging (WAL) mode for atomic writes, foreign key relational integrity, and durable storage of rooms, vector action logs, group chat history, immutable milestone snapshots, and user account records.

2.3 Physical Subnet Addressing & UDP Peer Discovery

To eliminate manual IP entry for students on the same Wi-Fi or Ethernet switch, the server integrates a UDP subnet discovery beacon on port 45455 / 41234. The server periodically inspects host physical network adapters (filtering out WSL, Hyper-V, Docker, and loopback virtual adapters) and broadcasts discovery beacons containing the active room codes, room titles, host name, and server port. Client applications listen on the discovery port, instantly displaying discovered active whiteboards with a 1-click 'Join Now' button.

2.4 Internet Bridging & Tunneling Architecture

While engineered for air-gapped LANs, the architecture incorporates an optional secure reverse tunnel gateway (via Cloudflare / LocalTunnel). When internet collaboration is requested, the server establishes an outbound TLS tunnel to the gateway. External peers connect via an encrypted HTTPS/WSS URL (e.g. https://whiteboard-room.loca.lt), achieving full cross-network parity without modifying firewall rules or performing router port-forwarding.

Subsystem	Component	Architectural Responsibility
Client GUI	React 18 + Tailwind CSS	Responsive ribbon toolbar, chat drawer, account logs, instances modal, grid selector
Canvas Engine	HTML5 Canvas 2D Context	60 FPS affine render loop, point interpolation, shape math, marquee selection, vector eraser
Communication	ws (RFC 6455) + dgram (UDP)	Sub-5ms WebSocket event stream + LAN broadcast beacon for zero-configuration discovery
Backend Host	Node.js + Express REST API	HTTP static asset serving, room validation, authentication, and sequence arbitration
Database Engine	SQLite (node:sqlite WAL)	Zero-configuration embedded relational storage: action log, milestones, audit history
Single-Click Executable	Collaborative_Whiteboard.exe	Compiled C# desktop wrapper providing seamless 1-click execution of entire application

3. Socket Communication and Application-Level Protocol

3.1 Full-Duplex WebSocket Implementation over TCP/IP

Communication is built on the standard WebSocket protocol (RFC 6455) running over stream-oriented TCP/IP sockets. The underlying transport layer disables Nagle's buffering algorithm (socket.setNoDelay(true)), ensuring coordinate packets are dispatched immediately without buffering delays. Unlike HTTP request-response polling, WebSockets maintain persistent, bi-directional channels allowing the server to push canvas actions and chat events to all connected clients with minimal transmission overhead (2-byte framing overhead).

3.2 Application-Level Wire Protocol Specification (JSON Schemas)

All application messages are framed as UTF-8 encoded JSON objects containing a mandatory type field. Below are the formal wire protocol schemas for key operations:

```
// 1. Client Join Request & Server State Hydration Acknowledgment Client -> Server: {"type": "join_room", "room_code":
"SPZG", "user_code": "usr_8f2a", "name": "Manas", "password": ""} Server -> Client: {"type": "join_ack", "success":
true, "room": {...}, "actions": [...], "chat_history": [...], "members": [...]} // 2. Vector Draw Action Broadcast
(Monotonically Sequenced by Server) Client -> Server: {"type": "draw_action", "room_code": "SPZG", "action": {"id":
"act_91", "tool": "pen", "points": [...], "color": "#2563eb", "width": 3}} Server -> Peers: {"type": "draw_action",
"room_code": "SPZG", "action": {"id": "act_91", "seq": 142, "user_code": "usr_8f2a", ...}} // 3. Remote Cursor
Telemetry (High-Frequency Subpixel Positioning) Client -> Server: {"type": "cursor_move", "room_code": "SPZG",
"user_code": "usr_8f2a", "name": "Manas", "x": 450.5, "y": 312.0} Server -> Peers: {"type": "cursor_move",
"room_code": "SPZG", "user_code": "usr_8f2a", "name": "Manas", "x": 450.5, "y": 312.0} // 4. Group Chat Broadcast &
Database Persistence Client -> Server: {"type": "chat_message", "room_code": "SPZG", "user_code": "usr_8f2a", "name":
"Manas", "message": "Reviewing schema"} Server -> Peers: {"type": "chat_message", "room_code": "SPZG", "message_id":
88, "seq": 143, "timestamp": "19:45:10", ...} // 5. 4-Dimensional RBAC Matrix Permission Update Host -> Server:
{"type": "rbac_change", "room_code": "SPZG", "dimension": "drawing", "policy": "host_only", "allowed_users": []}
Server -> All: {"type": "rbac_change", "room_code": "SPZG", "dimension": "drawing", "policy": "host_only"} // 6.
Point-in-Time Milestone Snapshot & Live Sandbox Broadcast Host -> Server: {"type": "milestone_create", "room_code":
"SPZG", "title": "Base Diagram", "actions": [...]} Host -> Server: {"type": "live_sandbox_broadcast", "room_code":
"SPZG", "actions": [...], "is_live": true} Server -> Peers: {"type": "live_sandbox_broadcast", "room_code": "SPZG",
"host_sandbox_actions": [...], "is_live": true}
```

4. Real-Time Synchronization and Eraser Engine

4.1 Synchronization Model: Optimistic Local Render with Server Arbitration

To achieve instantaneous responsiveness with zero perceptual lag, the client implements **Optimistic Local Rendering**: when a user drags their mouse or stylus, strokes are painted immediately onto the local HTML5 canvas. Simultaneously, coordinate points are bundled and dispatched asynchronously to the server. The central server acts as the authoritative serializer: it assigns each incoming action a strictly increasing monotonic integer sequence number (seq), commits the action to the SQLite database, and broadcasts it to all other peers in the room. Peers integrate the action into their local action lists, guaranteeing eventual consistency across all nodes.

4.2 Conflict-Free User-Scoped Undo/Redo Stacks

A common flaw in collaborative whiteboards is global undo: if Student A presses Undo, another user's recent stroke might mistakenly be erased. Our engine implements **User-Scoped Undo Stacks with Tombstone Markers**:

• When a user presses Undo (or Ctrl+Z), the client searches the action history backwards for the latest active action created by that specific user_code.
• An action_undo event with the target action ID is sent to the server.
• The server marks the record with a tombstone (undone: 1 in SQLite) and broadcasts the deletion ID.
• All connected clients immediately suppress rendering for that element ID while preserving unrelated peer strokes.

4.3 High-Performance Vector Element Eraser Engine

Traditional paint programs implement erasing via destination-out composite stroke masks. In vector whiteboards, masking creates heavy transparent layers that bloat payload sizes and fail to delete underlying geometric objects. Our application implements a true **Real-Time Geometric Vector Hit-Testing Eraser**:

• **Point-to-Segment Orthogonal Distance Math:** For freehand paths, when the user drags the eraser, the engine calculates the minimum perpendicular distance from the eraser coordinate (x, y) to each consecutive line segment (p1, p2) of every vector path:

d = |(p2.x - p1.x)(p1.y - y) - (p1.x - x)(p2.y - p1.y)| / sqrt((p2.x - p1.x)^2 + (p2.y - p1.y)^2)

• **Dynamic Hit Threshold:** If the distance is within the eraser radius (radius + stroke_width / 2), the element is flagged as hit.
• **Bounding Box Collision:** For geometric shapes (rectangles, ellipses, tables, text), axis-aligned bounding box (AABB) intersection testing is executed.
• **Tombstone Elimination:** The touched element is immediately marked undone: true, synchronized across all peers via WebSocket, and removed from active canvas redraw iterations.

4.4 Selection Marquee & Group Translation Math

The Selection Tool supports click-and-drag bounding marquee selection. As the user drags the cursor, a translucent selection box is rendered. Upon release, all vector actions whose coordinates fall within the marquee bounds are highlighted with selection corner pills. Dragging the selection calculates displacement (dx, dy), updating all enclosed vector coordinates atomically and broadcasting the updated positions across the network.

5. Concurrency Handling, State Management and Recovery

5.1 Concurrency Architecture in Node.js and SQLite

The server utilizes Node.js's event-driven, single-threaded asynchronous runtime. Because event processing runs on a single thread, socket message dispatching is inherently free from race conditions common in multi-threaded socket servers. For database persistence, SQLite is operated in **WAL (Write-Ahead Logging)** mode. Multi-statement transactions (such as room creation, milestone restoration, and action sequence commits) are executed using BEGIN IMMEDIATE transactions, preventing database locks or corrupted transaction states.

5.2 Vector Representation vs Bitmap Snapshots

Commercial whiteboards that transmit periodic raster image frames (PNG/JPEG) suffer from massive network payload sizes (>500 KB per update) and irreversible loss of individual stroke editability. Our system represents the canvas entirely as discrete mathematical vector entities (stroke coordinates, bounding boxes, styling attributes, and author signatures). Updates average under 150 bytes per stroke, allowing smooth 60 FPS drawing even over congested subnets.

Property	Data Type	Description & Role in Vector Synchronization
id	String	Globally unique identifier (e.g. 'act_179451234') used for undo/redo and erasing
tool	String	Vector primitive: 'pen', 'highlighter', 'rect', 'circle', 'arrow', 'table', 'text'
points	Array[Point]	Array of subpixel coordinates [{x, y}, ...] representing vector path vertices
color / width	String / Int	HEX color string (e.g. '#2563eb') and stroke thickness in virtual canvas pixels
fillColor	String / null	Interior polygon fill color (or null for transparent wireframes)
user_code	String	Author identifier enforcing user-scoped undo and RBAC permission validation
undone	Boolean / Int	Tombstone flag: 1 if deleted/undone, 0 if actively rendered on canvas
seq	Integer	Monotonic sequence integer assigned by server to enforce total ordering

5.3 Late-Joiner State Hydration & Mid-Session Recovery

When a student connects to an ongoing room session, the server executes **Historical State Hydration**. The server queries the SQLite database for all active vector actions (WHERE undone = 0 ORDER BY seq ASC), all historical chat messages, the participant roster, and the current RBAC permissions matrix. These are bundled into a single join_ack payload. The client's React canvas engine parses the array and executes a fast replay, reconstructing the identical whiteboard state within 50 milliseconds.

5.4 Network Fault Tolerance, Heartbeats & Clean Cleanup

Local Wi-Fi drops, laptop lid closures, or sleep events can terminate TCP connections abruptly without FIN packets. The system implements multi-layer recovery:

• **Ping-Pong Heartbeats:** The server issues ping heartbeats every 15 seconds. Sockets failing to respond within 30 seconds are terminated cleanly.

• **WeakMap Metadata & Memory Pruning:** Connection references are stored in WeakMap and isolated room sets. Upon socket closure, the server broadcasts user_left with an updated member roster and removes stale remote cursors.

• **Automatic Client Reconnection:** If connection drops, the client automatically attempts reconnection with exponential backoff.

6. Security, Role-Based Access Control (RBAC) and Governance

6.1 Role-Based Access Control (RBAC) Hierarchy

To maintain orderly academic collaboration during laboratory demonstrations and multi-student exercises, the system implements an explicit **Role-Based Access Control (RBAC) Hierarchy** across three privilege tiers:

Role Tier	Assignment	Privileges & Operational Capabilities
Host (Presenter)	Room Creator	Full drawing rights; configure 4D RBAC permissions; lock room; kick users; manage milestones; live sandbox broadcast; clear board.
Co-Host	Promoted by Host	Elevated privileges: drawing in locked mode; manage participant permissions; moderate chat.
Participant	Default on Join	Standard drawing rights (unless locked); send chat messages; local sandbox testing; raise hand; trigger emoji reactions.

6.2 Deterministic 4-Dimensional RBAC Matrix

Unlike basic tools with a single 'view-only' toggle, our system enforces a **4-Dimensional Permission Matrix**:
1. Canvas Drawing: Controls who can draw, erase, or modify vector elements on the shared canvas.
2. Group Chat: Controls who can send synchronized messages to the in-app chat drawer.
3. Instance Management: Controls who can create milestone snapshots and private sandbox simulations.
4. Public Works Sharing: Controls who can publish whiteboard instances to the shared room public gallery.
Each dimension can be independently configured by the Host into three policy states: **Everyone**, **Host Only**, or **Selected Members** (via a multi-select modal dialog). Enforcement is dual-layered: the client hides tools and displays a lock banner, while the server validates incoming packet signatures and drops unauthorized events.

6.3 Milestone Snapshots & Isolated Sandbox Simulation

A standout feature of the application is the **Milestone & Isolated Sandbox Engine**:
• **Point-in-Time Milestones:** The host can snapshot the entire canvas at critical design junctures (e.g. 'Sprint 1 Architecture').
• **Private Sandbox Mode:** Participants can toggle into Sandbox Simulation Mode. The canvas background shifts to dark mode with an orange status banner. Users can freely test, sketch, and experiment without altering the live collaborative board.
• **Host Live Sandbox Broadcast:** When the host tests ideas in their sandbox, they can toggle 'Go Live with Sandbox'. Peers can view the host's private sandbox in real time on a dedicated viewing channel without corrupting the production board.
• **Atomic Restore:** The host can restore any milestone or sandbox state back onto the live canvas with a single click.

6.4 Integrated Group Chat & SQLite Relational Persistence

The side-drawer chat interface allows synchronous technical discussion alongside drawing. Chat messages are delivered over the primary WebSocket channel, stamped with server timestamps, and saved to the SQLite chat_messages table. This ensures historical continuity: late joiners immediately receive the full chat thread alongside the whiteboard drawings.

7. Comprehensive User Guide

7.1 Primary 1-Click Execution Model (Collaborative_Whiteboard.exe)

The software provides a frictionless, zero-terminal user experience. Users are **never required to open terminal windows or type command lines**:

1. **Single-Click Launch:** Simply double-click **Collaborative_Whiteboard.exe** in the root application directory.
2. **Silent Self-Contained Initialization:** The executable silently boots the background Node.js WebSocket engine on port 5000 (using the bundled runtime) and immediately opens the full-screen desktop whiteboard interface in front of the user.
3. **Unified Home Lobby:** From this single application window, users can directly:
   - **Host a Whiteboard:** Click 'Create Room' to generate an active session code (e.g. SPZG).
   - **1-Click Auto-Join:** Whiteboards hosted on the same Wi-Fi/LAN automatically appear under 'Active Whiteboards Discovered on Same LAN Subnet' with a green 'Join Now' card.
   - **Join via Code:** Enter a 4-letter room code and name to join directly.
4. **Automatic Clean Exit:** Closing the application window automatically shuts down all background server services cleanly.

7.2 Academic Modular Executables (Server & Client Separation)

To fulfill the formal course deliverables specification, two modular standalone executables are also provided:

- **Server_Application.exe [Deliverable 1 - Server Application]:** A dedicated executable to run on a machine in the LAN to host and manage collaborative whiteboard sessions. It boots the backend, displays the active LAN IP, port 5000, and discovery beacons, and offers keyboard controls ([O] to open browser, [Q] to quit).
- **Client_Application.exe [Deliverable 2 - Client Application]:** A standalone executable that users run on client workstations to join sessions. Provides a connection dialog to enter the host LAN IP or click 'Join Local'.

7.3 Air-Gapped LAN vs. Internet Collaboration

- **Offline Air-Gapped Mode:** Connect host and client devices to the same Wi-Fi router, Ethernet switch, or mobile hotspot. No WAN Internet access is required. Clients can join using the 4-letter room code, the green auto-discovery card, or by entering the host LAN IP (e.g. http://192.168.1.15:5000) in any web browser.
- **Internet Cross-Network Mode:** When collaborating outside the local network, an outbound secure tunnel (Cloudflare / LocalTunnel) provides a public share link (e.g. https://my-room.loca.lt), achieving 100% feature parity.

7.4 Complete Canvas Tools and Features Reference

Tool / Feature	Shortcut	Operational Instructions & Capabilities
Freehand Pen	Pen (P)	Smooth vector drawing with custom line width (1px to 24px) and full hex color palette.
Highlighter	Highlighter (H)	Semi-transparent overlay highlighter for annotating diagrams and text without obscuring strokes.
Geometric Shapes	Shapes Gallery	Instant insertion of Rectangle, Circle/Ellipse, Triangle, Diamond, Star, Hexagon, or Arrow.
Selection Tool	Cursor (S)	Click and drag a marquee box across the canvas to multi-select items. Drag selected elements to move them.
Snap Screenshot	Camera / Crop	Click and drag a bounding rectangle over any canvas area to capture and download a high-res PNG.
Vector Eraser	Eraser (E)	Active geometric hit-tester. Drag over any vector stroke or shape to tombstone it permanently across peers.
Dynamic Table	Table Grid	Insert customizable data tables. Double-click cells to edit text, add/remove rows and columns dynamically.
Text & OCR	Text (T)	Place rich text labels with customizable typography (Inter, JetBrains Mono, Roboto, Caveat) & Ink OCR.
Milestone Sandbox	Flask / Beaker	Toggle into private Sandbox Simulation to test ideas safely. Host can broadcast live sandbox view to peers.
Group Chat	Chat Drawer (C)	Integrated real-time text chat with SQLite persistence, user avatar badges, and timestamps.
4D RBAC Controls	Shield / Lock	Host can restrict Drawing, Chat, Instances, or Public Works to 'Host Only', 'Selected Members', or 'Everyone'.

8. Experimental Evaluation, Benchmarks and Team Contributions

8.1 Physical LAN Experimental Testbed Setup

Empirical testing was conducted in a physical laboratory environment using four independent client laptops connected to an isolated 802.11ac Wi-Fi router (subnet 192.168.1.0/24) with the server hosted on a dedicated workstation (192.168.1.15). WAN internet access was physically disabled at the router uplink to rigorously verify 100% autonomous offline operation.

8.2 Quantitative Performance Benchmarks

Evaluation Metric	Measured Result	Target Threshold	Compliance Status
End-to-End Event Broadcast Latency	3.6 ms (average)	< 15.0 ms	✔ Exceeded Target
Peak Stroke Coordinate Throughput	180 packets / sec	> 60 packets / sec	✔ Smooth Spline Motion
Packet Delivery Reliability over LAN	100.0% (0 lost bytes)	100% (TCP stream)	✔ Zero Loss Guaranteed
Server Memory Footprint (4 clients)	34.2 MB RAM	< 150 MB	✔ Highly Lightweight
Client Canvas Frame Rate (active drawing)	59.8 – 60.0 FPS	≥ 30.0 FPS	✔ Fluid 60 FPS Render
Offline Subnet Discovery Latency	42 ms (average)	< 500 ms	✔ Instant Room Beacon

8.3 Detailed Individual Team Member Contributions

Team Member	Assigned Role	Key Implemented Contributions & Deliverables
CS24B1025 **Anirudh**	Network Protocol & Server Architecture	• Engineered Node.js WebSocket server pipeline and room connection isolation maps. • Implemented UDP physical network adapter scanner and subnet discovery broadcast. • Designed TCP_NODELAY optimization achieving sub-5ms event dispatching. • Authored Server_Application.exe standalone launcher with LAN IP reporting.
CS24B1097 **Krishna Chaitanya**	Canvas 2D Engine & Vector UI	• Engineered React 18 Canvas 2D engine with 2D affine transforms (zoom 15%–400%, pan). • Implemented geometric hit-testing vector eraser eliminating destructive stroke masks. • Designed click-drag marquee selection and group movement translation math. • Built snap area screenshot crop tool with instant PNG export.
CS24B1108 **Harish**	Group Chat, Persistence & Room Isolation	• Engineered SQLite WAL embedded database schema with atomic transaction rollback. • Built docked side-drawer group chat with avatar badges and timestamps. • Implemented 4-letter alphanumeric session codes and PIN authentication. • Developed account management modal and user session audit logging.
CS24B1106 **Manas Vivek**	Concurrency Control, RBAC & Sandboxing	• Engineered 4-dimensional RBAC matrix (Canvas, Chat, Instances, Public Works). • Developed point-in-time milestone snapshots and isolated sandbox simulation. • Implemented host live sandbox broadcast and atomic canvas restoration. • Configured physical LAN testbed and authored empirical benchmark evaluation.

8.4 Conclusion

The project rigorously demonstrates that an air-gapped, zero-cloud multi-user collaborative application can achieve industry-standard responsiveness, deterministic concurrency control, and comprehensive vector editing capabilities over standard local subnets using modern WebSockets, React 18, and SQLite. The standalone executables deliver true single-click, zero-terminal ease of use, fulfilling all course deliverable objectives.